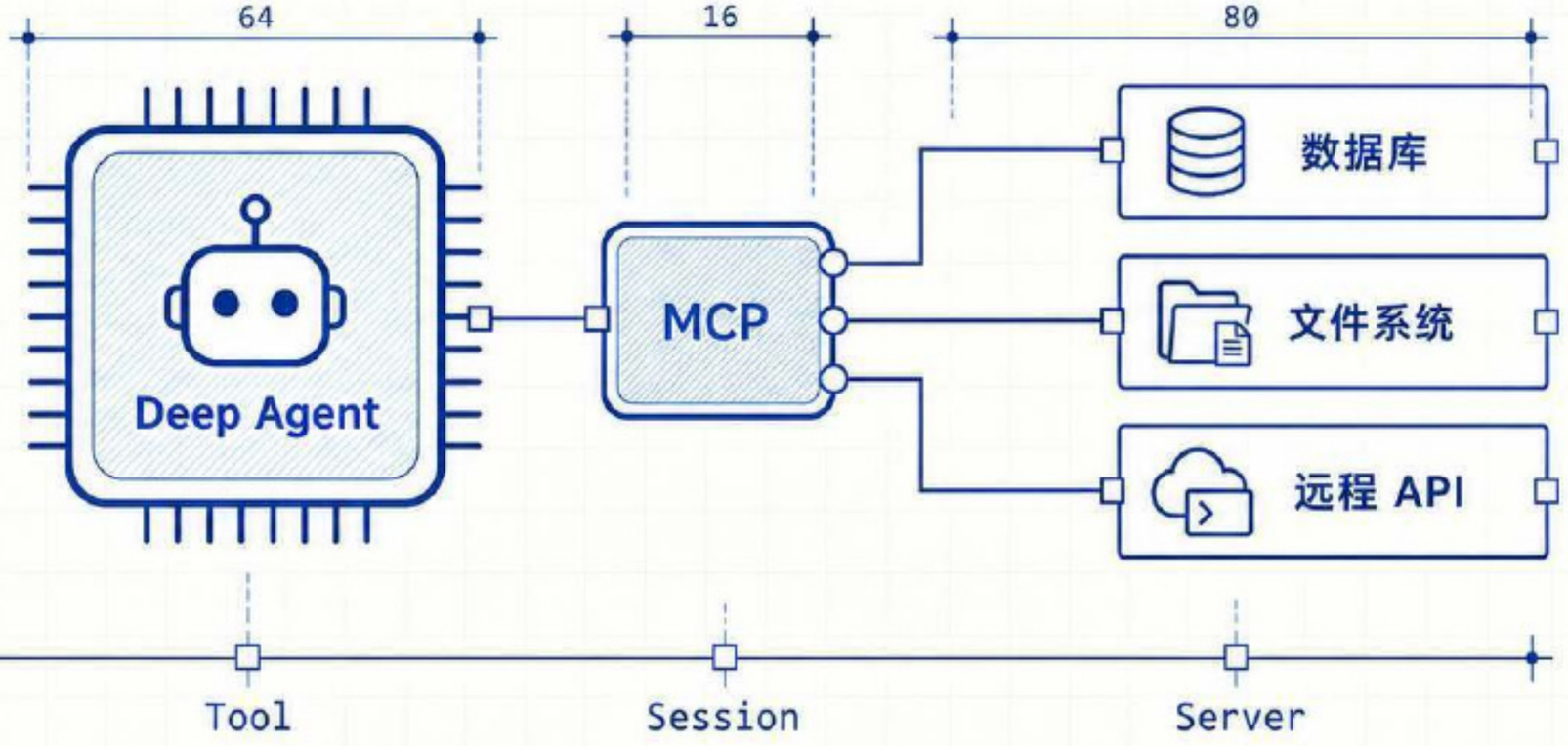
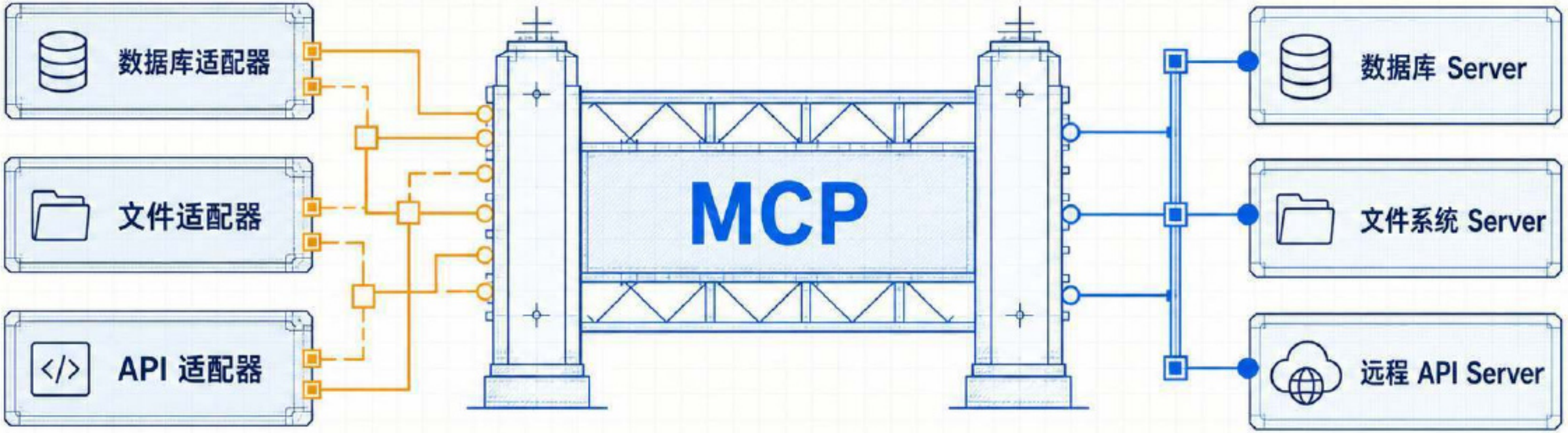
Deep Agents 实战

第 16 讲：MCP — 用标准协议扩展工具生态



专用适配越多，工具生态越难维护

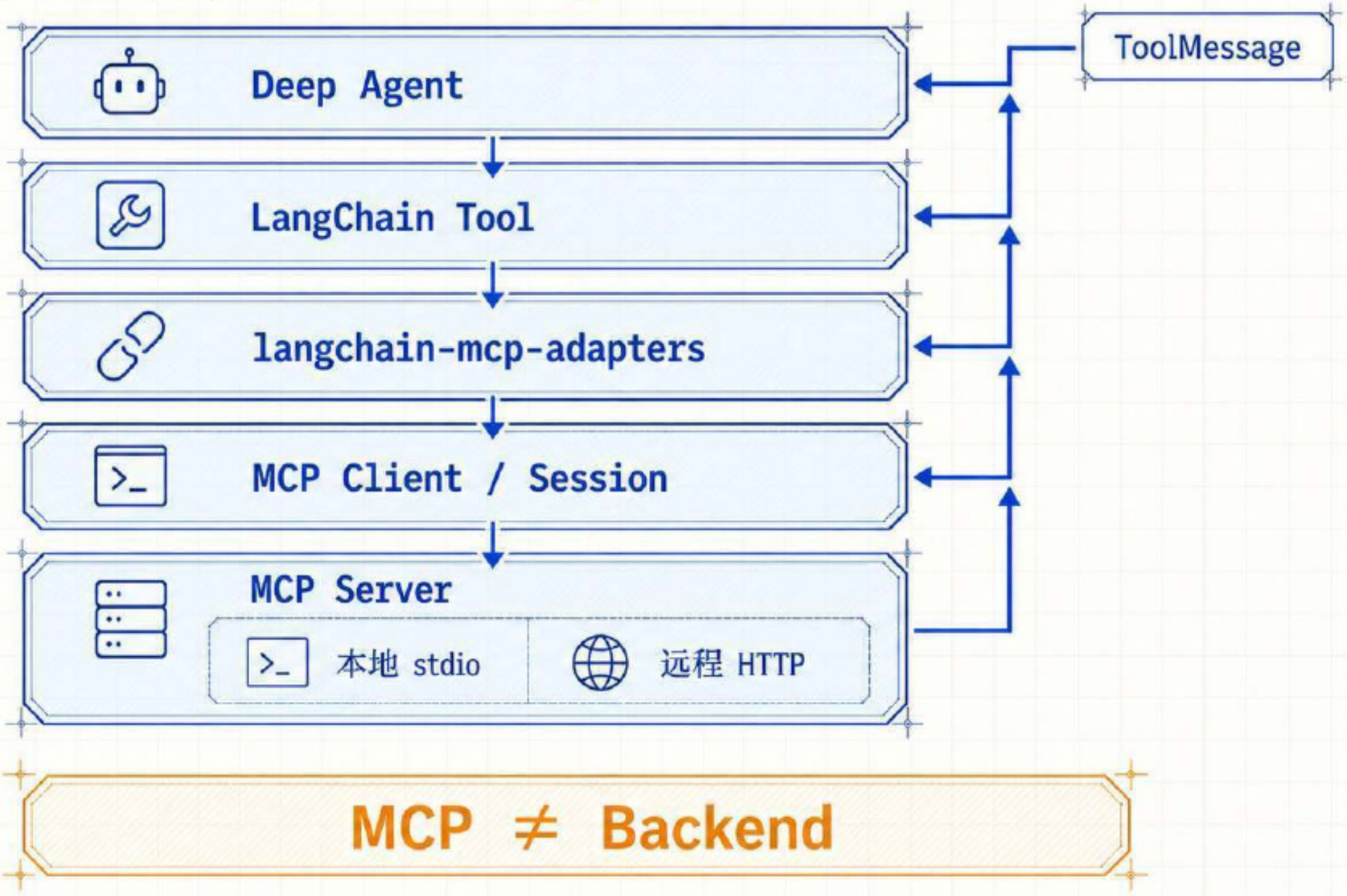
MCP 把数据库、文件系统与 API 收敛到同一协议边界





MCP 不是 Backend，它位于 Agent 与外部能力之间

五层职责清晰，结果沿反向路径变成 ToolMessage



tools= 只接 Tools，不会自动接入另外两类能力

Resources 与 Prompts 都需要应用主动放入上下文



十几行 FastMCP 代码，就能把函数变成工具

函数名、docstring 与类型标注分别生成 name、description、inputSchema

```python
from mcp.server.fastmcp import FastMCP

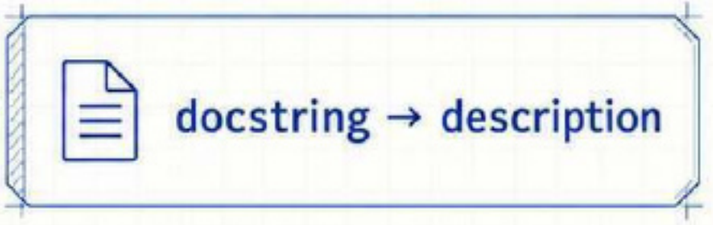
mcp = FastMCP("Chapter 12 Math")

@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two integers exactly."""
    return a + b

@mcp.tool()
def multiply(a: int, b: int) -> int:
    """Multiply two integers exactly."""
    return a * b

if __name__ == "__main__":
    mcp.run(transport="stdio")
```









不要先接模型，先逐层证明协议链路

模型与 API Key 暂时都不需要



无模型冒烟测试：先发现，再直接调用

以下为异步函数内片段；完整导入和 Client 配置见配套源码

异步函数内片段

```
tools = await client.get_tools()
```

```
add_tool = next(
    t for t in tools
    if t.name == "course_math_add"
)
```

```
result = await add_tool.ainvoke({"a": 37, "b": 58})
```

```
text = next(
    b["text"] for b in result
    if b["type"] == "text"
)
```

```
assert text == "95"
```

关键检查点

① Client 使用 sys.executable 与 Server 绝对路径

② 开启 tool_name_prefix=True，核对名称与参数 Schema

③ 直接 ainvoke；内容块不是裸整数

⚠ 预期结果 37 + 58 = 95；这一步不使用模型密钥。实际协议运行仍需本地 MCP 依赖。

异步调用与错误：分清三个通道

核对 langchain-mcp-adapters 0.3.2

成功	`await tool.ainvoke(...)`	内容块列表
工具执行错误	`CallToolResult(isError=True)`	`ToolMessage.status="error"`
基础设施失败	传输 / Session / 内容转换	抛出异常

使用 tool.ainvoke 与 agent.ainvoke；同步 invoke 不受支持。

handle_tool_errors=False 时，Server 执行错误改抛 ToolException。

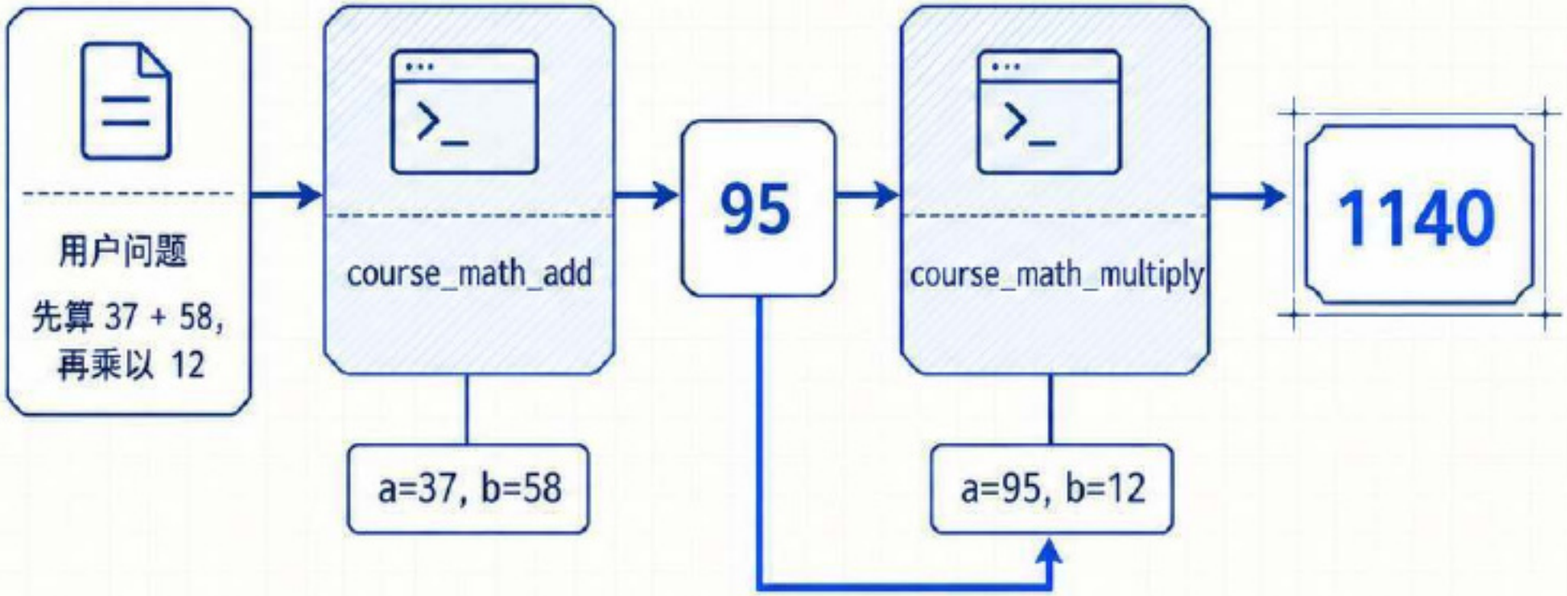
create_deep_agent 本身仍是同步构造函数。

协议通过后，Deep Agent 只需要接收工具列表

正确性应检查工具序列，而不是模型措辞

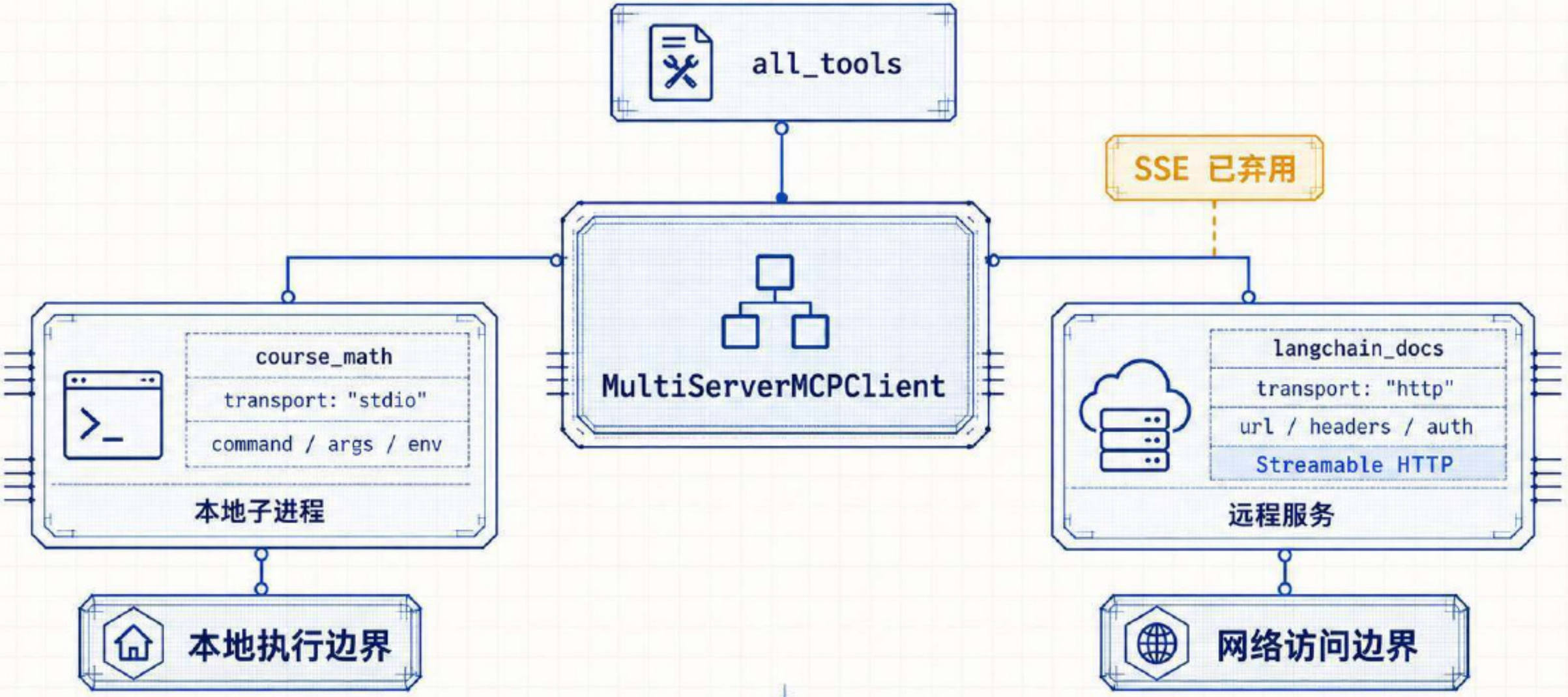
```
tools = await client.get_tools()

agent = create_deep_agent(model=model, tools=tools)
```



一个 Client 可以同时编排本地与远程 MCP Server

stdio 管本地子进程，Streamable HTTP 管远程服务



工具名前缀稳定命名，审批使用最终名称

凭证由应用管理，不能让模型填写 Token

命名与审批

Server 名
billing

+

原始工具名
charge_card

tool_name_prefix=True

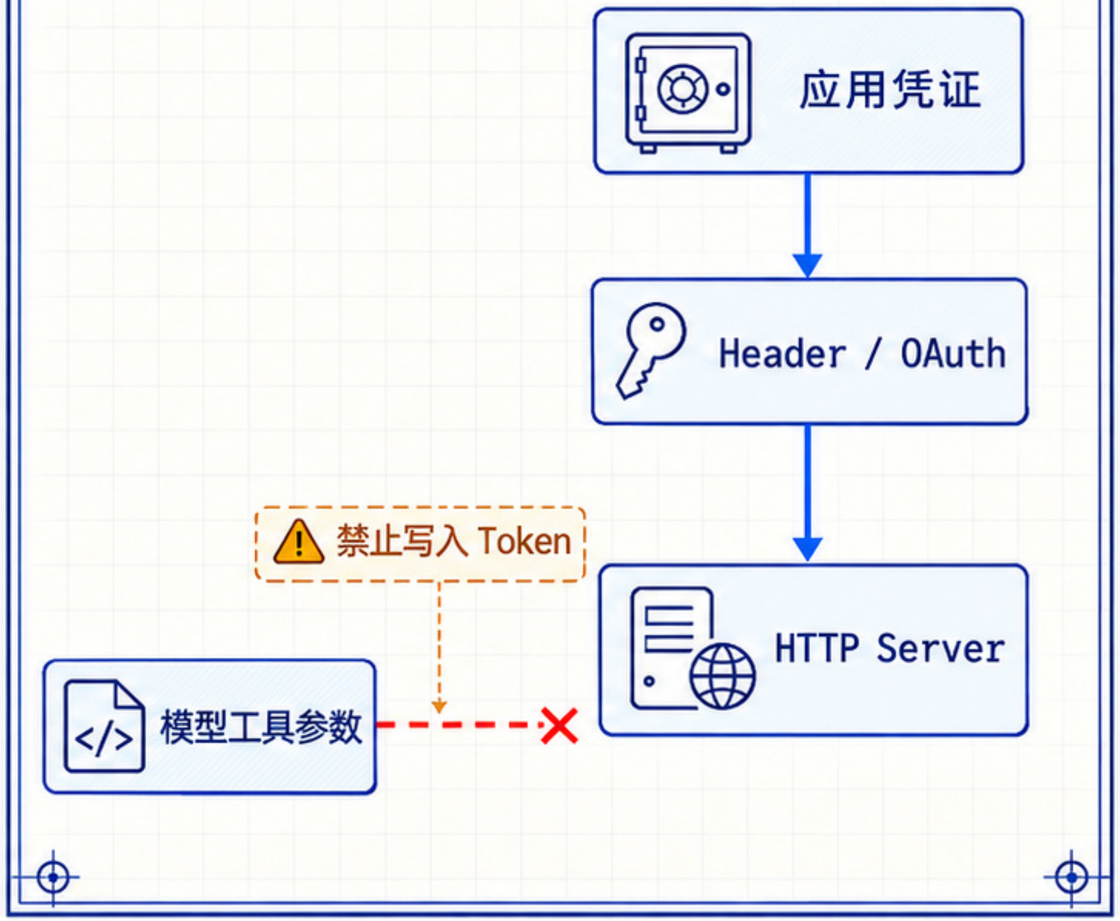
<server_name>_<tool_name>

最终工具名
billing_charge_card

```
interrupt_on={
   "billing_charge_card": {
     "allowed_decisions": ["approve", "reject"]
   }
}
```

配置片段；需 checkpointer，同一 thread_id 恢复

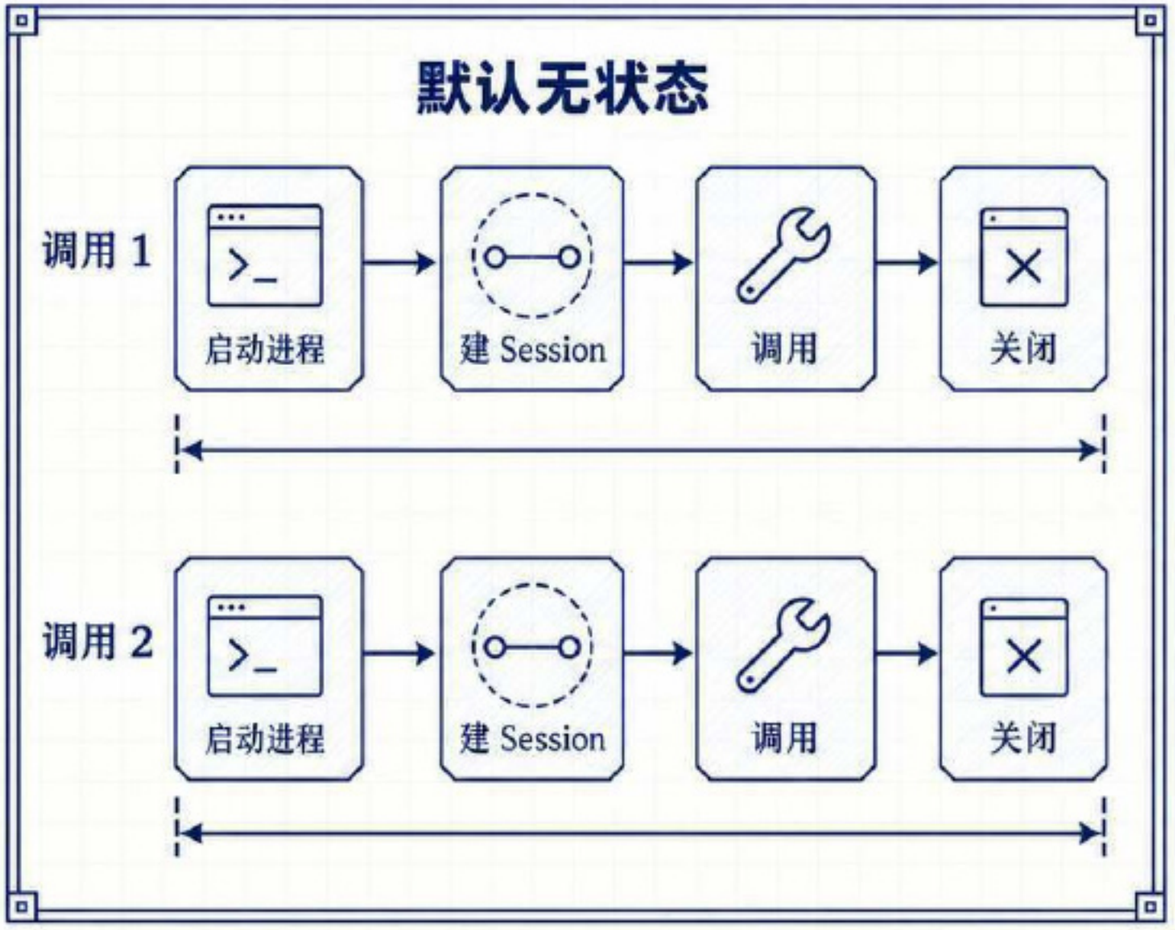
认证边界

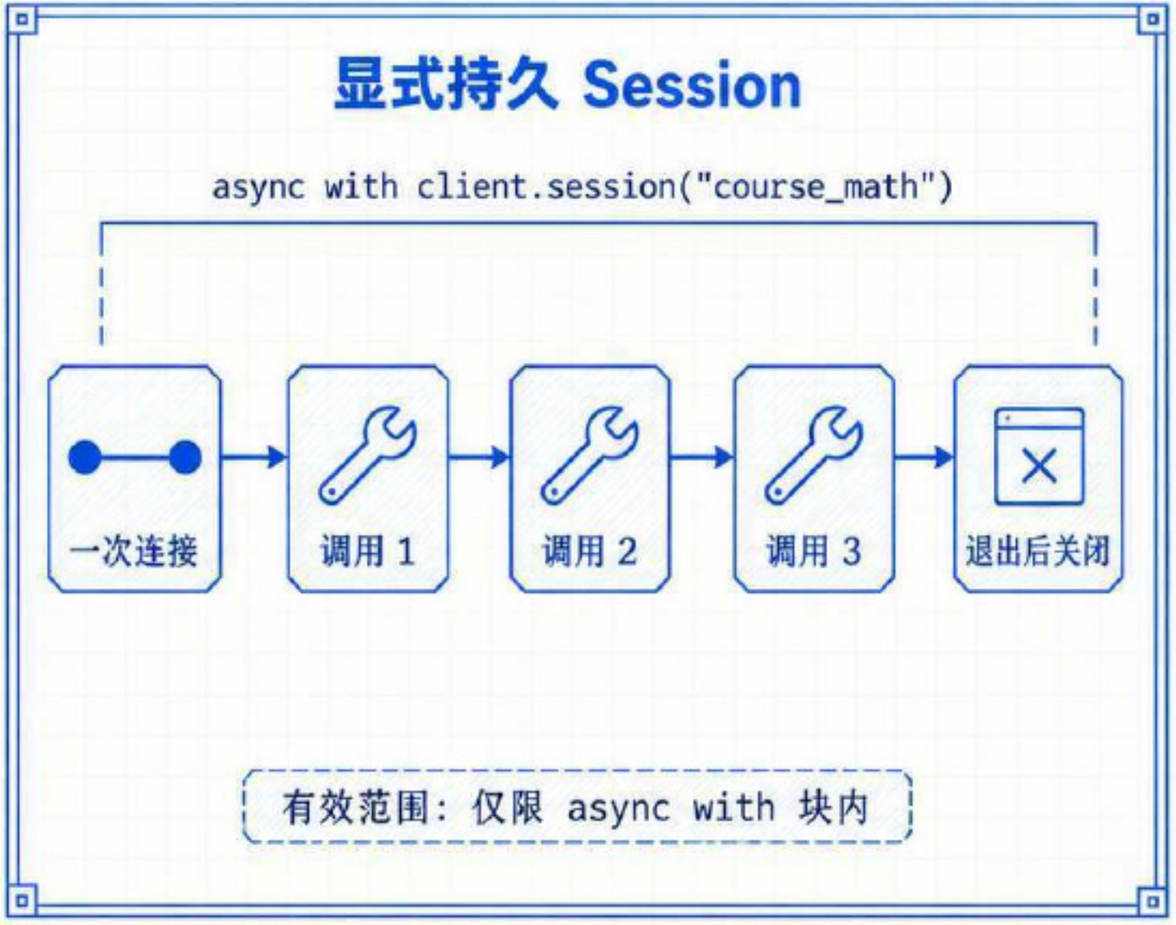


⚠ destructiveHint 等 annotation 不会自动生成审批规则；前缀降低冲突风险，仍需检查名称唯一性。

默认每次调用都重建 Session，持久模式才会复用

两种模式的隔离、成本与状态语义不同





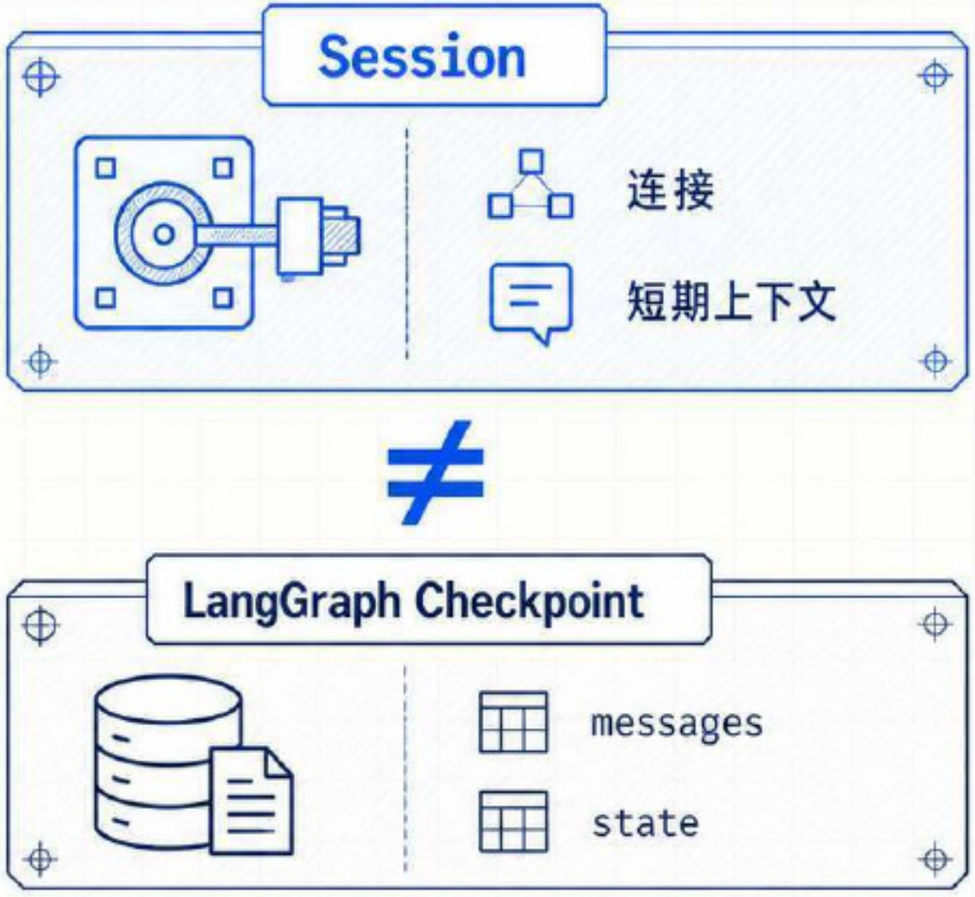
Session 复用连接，Checkpoint 保存 Agent 状态

暂停、重启与 HITL 恢复不能依赖进程内存

持久 Session

```python
async with client.session("course_math") as session:
    tools = await load_mcp_tools(
        session,
        callbacks=client.callbacks,
        tool_interceptors=client.tool_interceptors,
        server_name="course_math",
        tool_name_prefix=client.tool_name_prefix,
        handle_tool_errors=client.handle_tool_errors,
    )
```

生命周期边界



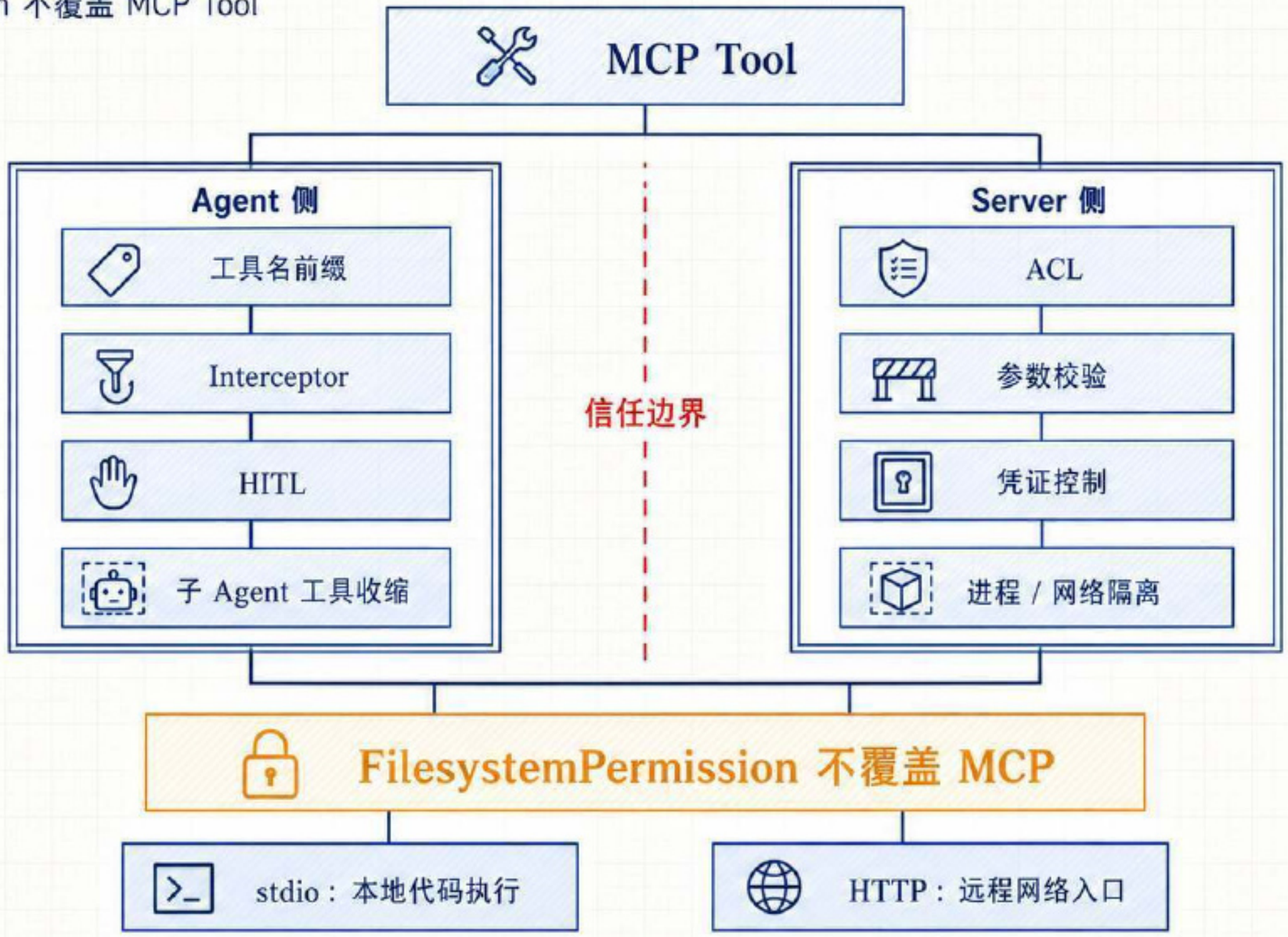


Server DB

长期业务状态 · 幂等写入

MCP 工具要同时守住 Agent 侧与 Server 侧

FilesystemPermission 不覆盖 MCP Tool



从教学示例到可复现工程

不要把脚手架启动成功当成集成验证

固定环境

- Python 3.11+
- deepagents >=0.7,<0.8
- langchain-mcp-adapters >=0.3,<0.4
- mcp >=1.28,<2

⚠ 提交 lockfile 或保存环境快照

三个文件

- math_server.py：本地 Server
- check_mcp.py：无模型协议检查
- agent.py：模型与工具集成

分层验收

- Server 函数与 Schema
- 直接工具调用 37 + 58 = 95
- 模型 Trace：95 × 12 = 1140

⚠ 检查真实调用，不固定自然语言措辞

⚠ AgentSeek deepagents/mcp 可作脚手架；模板、传输与依赖能力应在所用版本重新核验。

结构化结果不等于模型可见文本

Tools、Resources、Prompts 的返回通道各不相同

Tool 结构化结果

structuredContent

↓

ToolMessage.artifact["structured_content"]

↓

供应用读取；默认不自动追加到模型文本

Resources

get_resources() → Blob

↓

应用决定检索、写入文件
或注入上下文

↓

⚠ 动态模板需具体 URI

Prompts

get_prompt() → 消息列表

↓

应用决定放入哪段对话

↓

⚠ 0.3.x 转换仅接受文本

⚠ 若模型需要结构化字段，在 *Interceptor* 中选择性转换；不要把大型 JSON 无条件复制到历史。

Interceptor 与子 Agent：补齐授权边界

Client 侧检查不能取代 Server 侧授权

Client Interceptor

`request.server_name = "billing"` + `request.name = "charge_card"`

这里使用原始 MCP 工具名

注入可信身份、审计、拒绝或转换结果

⚠ 持久 Session 不因修改 Header 自动重新认证

子 Agent 工具收缩

- 显式 tools 替换继承的额外工具
- 不会自动移除 Harness 文件工具或 execute
- 声明式 interrupt_on 显式配置会整体替换
- 预编译与远程子 Agent 须自行配置审批

Server 再做 ACL 和参数校验；stdio 是本地代码执行，HTTP 需要 TLS、主机限制与秘密管理。

先证协议链路，再接模型、会话与权限

Server 函数 → MCP 发现 → LangChain Tool → Deep Agent → 生产控制

无密钥冒烟测试越早变绿，后续问题越容易定位

